

High Performance Computing Assignment

Performance Optimisation of a Parallel Stencil Solver

Giacomo Serafini
15 July 2026

1 Introduction

1.1 Problem Description

Stencil computation is one of the fundamental computational kernels in high-performance computing (HPC), widely employed in the numerical solution of partial differential equations. In this work, the focus is on a two-dimensional five-point stencil. For each grid point, the updated value depends on the current value of the point itself together with its four direct neighbours (north, south, east, and west). At every iteration, a new grid is computed from the values of the previous iteration, making each update independent within the same time step while introducing dependencies between consecutive iterations:

$$new_{i,j} = \alpha \cdot old_{i,j} + \frac{1 - \alpha}{4} \cdot (old_{i-1,j} + old_{i+1,j} + old_{i,j-1} + old_{i,j+1})$$

Since the computation performed at each grid point is relatively simple, stencil applications are often memory-bound rather than compute-bound. For this reason, performance is strongly influenced by the efficiency of memory accesses, cache reuse, and data movement. Improving serial performance therefore requires optimizations that maximize spatial and temporal locality while minimizing unnecessary memory traffic before parallelization is considered.

The structured nature of stencil computations naturally exposes a large amount of parallelism because grid points belonging to the same iteration can be updated concurrently. In a shared-memory environment, this parallelism can be exploited using OpenMP by distributing iterations of the computational loops among multiple threads. Since all threads operate on a common memory space, communication is implicit.

Scaling the computation across multiple compute nodes requires a distributed-memory approach.

In this case, the computational domain is partitioned among different MPI processes, each responsible for a subdomain of the global grid. Since the update of boundary points depends on neighbouring subdomains, adjacent processes must exchange boundary (halo) cells after each iteration.

1.2 Hardware Platform

The experiments presented in this report were conducted on the ORFEO high-performance computing cluster, specifically on the EPYC partition, whose compute nodes are equipped with two AMD EPYC 7H12 64-core processors, for a total of 128 CPU cores and 512 GiB of DDR4 memory per node. Each processor is organized into four NUMA domains (eight per node) and features a hierarchical cache consisting of 32 KiB L1 data cache and 512 KiB L2 cache per core, together with 16 MiB of shared L3 cache per core complex (512 MiB total per node). The nodes are interconnected through a high-speed InfiniBand network with RDMA support.

The experiments were performed using the software environment provided by the cluster, consisting of *GCC 14.3.1* and *OpenMPI 4.1.6* with OpenMP support. The applications were compiled through the MPI compiler wrapper (`mpicc`) using GCC, with optimization flags `-O3` and `-march=native`, and executed using the cluster-provided runtime environment.

2 Serial Optimisation

2.1 Baseline Implementation

The starting point is a direct implementation of the five-point stencil update. The domain is stored as a single flattened 1D array of doubles (`plane_t.data`) of size $(sx + 2) \times (sy + 2)$, where the +2 padding on each dimension holds the

halo/boundary frame. Indexing is done through an $\text{IDX}(i, j) = j \times \text{fxsize} + i$ macro so that the grid is addressed row-major. The update itself is a double nested loop over the interior ($sx \times sy$) points, computing

$$\begin{aligned} \text{new}_{i,j} &= \alpha \text{old}_{i,j} + \\ \alpha_n (\text{old}_{i-1,j} &+ \text{old}_{i+1,j} + \text{old}_{i,j-1} + \text{old}_{i,j+1}) \end{aligned}$$

with α and α_n precomputed as constants outside the loop rather than recomputed per point. Old and new planes are stored as two separate buffers and their roles are swapped at the end of each iteration.

2.2 Optimisations

The stencil update performs only a small amount of arithmetic for each memory access, making the stencil inherently memory-bound. Considering a single update in isolation, each grid point requires five double-precision loads (the center point and its four neighbours) and one double-precision store, corresponding to approximately 48 bytes of memory traffic. The computation itself consists of one multiplication for the center coefficient, four additions to accumulate the neighbouring values, one multiplication for the neighbour contribution, and one final addition, for a total of roughly six floating-point operations. This yields an arithmetic intensity of

$$AI \approx \frac{6 \text{ FLOPs}}{48 \text{ B}} = 0.125 \text{ FLOPs/B}$$

As we can see, performance is primarily limited by the rate at which data can be supplied from the memory hierarchy rather than by the available floating-point throughput. This observation can also be quantified by comparing the computational capability of the processor with the throughput required by the stencil computation. An AMD EPYC 7H12 core provides a theoretical peak double-precision performance of approximately 42 GFLOP/s, while the available memory bandwidth corresponds to roughly 1.6 GB/s per core when the node bandwidth is shared among all 128 cores. Given the arithmetic intensity of the stencil ($AI \approx 0.125 \text{ FLOPs/B}$), only

$$1.6 \times 0.125 \approx 0.20 \text{ GFLOP/s}$$

of computational throughput can be sustained before becoming limited by memory bandwidth. Essentially, the processor is theoretically capable of

performing more than 200 times the amount of computation that the memory subsystem can feed for this update. This large imbalance confirms that optimizing memory accesses and cache reuse is considerably more beneficial than attempting to reduce the computational cost of the stencil update.

For this reason, the first optimization efforts focused on improving data locality and cache utilization:

Cache-blocking (tiling). Although the baseline implementation already exhibits good spatial locality along the row direction, its temporal locality is limited. During a row-major sweep, each update requires values from the current row together with the rows immediately above and below, each $\text{fxsize} \times 8$ bytes away from the current cache line. For large grids, these rows cannot remain in the L1 cache throughout the entire sweep, causing data to be repeatedly fetched from lower cache levels or main memory. To improve cache reuse, the `update_plane_inside` function was rewritten using cache blocking (tiling). The computational domain is partitioned into square tiles, and each tile is processed independently before moving to the next one. While an entire tile does not fit in the L1 cache, this is not required: only the working set corresponding to a few neighbouring rows of the current tile needs to remain resident. Consequently, data loaded for one stencil update are reused by subsequent updates before being evicted, significantly improving temporal locality and reducing cache misses. Several tile sizes, ranging from 32 to 512, were evaluated experimentally: a tile size of 64 provided the best overall compromise between execution time, parallel speedup, and cache efficiency. Compared to the untiled implementation, cache blocking reduced the L1 data cache miss rate from 8.4% to approximately 4.3%, nearly halving the number of L1 data cache misses.

Pointer aliasing. The input and output grids are stored in independent buffers. The pointers were declared using the `restrict` qualifier to explicitly inform the compiler that these memory regions do not overlap. This removes potential aliasing ambiguities and allows more aggressive compiler optimizations.

Memory alignment. Both planes are allocated using `posix_memalign(...,64,...)` instead of `malloc`, guaranteeing that the beginning of each allocated buffer is aligned to a 64-byte cache line boundary. This improves compatibility with SIMD vector loads and avoids unnecessary penalties associated with misaligned memory accesses.

Compiler-level optimisations. Several minor optimizations were applied to reduce unnecessary operations inside the hot loops. Constants such as stencil coefficients were computed once before entering the iteration space, avoiding repeated calculations for every grid point. Similarly, expensive operations such as divisions were replaced by equivalent multiplications.

3 OpenMP Parallelisation

3.1 Parallel Strategy

The shared-memory parallelisation was implemented using OpenMP. Since the update of each grid point depends only on values from the previous iteration, all interior points can be computed independently within the same time step. Therefore, the stencil update can be directly distributed among multiple threads without requiring explicit synchronization between them.

The computation is divided into two functions: `update_plane_inside`, which processes the interior region and represents the majority of the workload, and `update_plane_border`, which updates the boundary cells. Both updates are parallelised, although the interior update dominates the execution time.

3.2 Implementation

The interior computation is parallelised by distributing tiles among OpenMP threads:

```
#pragma omp parallel for collapse(2) \
    schedule(static)
for (tiles...)
```

The `collapse(2)` clause combines the two tile-index loops into a single iteration space, allowing a larger number of independent tasks to be distributed among threads. The `schedule(static)`

policy was selected because all tiles perform an equivalent amount of work; therefore, a predetermined distribution minimizes scheduling overhead while maintaining good load balance.

Inside each tile, the innermost loop is annotated with `#pragma omp simd` to explicitly allow vectorization of the stencil update. Combined with the aligned memory allocation and the absence of pointer aliasing introduced in the previous section, this enables the compiler to generate efficient SIMD instructions.

The border update follows the same parallel approach. Each boundary direction is processed through a separate parallel loop, while avoiding unnecessary synchronization between the different regions. Since the border contains only a small fraction of the total grid points, its contribution to the overall execution time is limited.

Thread Affinity and NUMA Locality The Orfeo compute nodes are based on a NUMA architecture, where memory access performance depends on the locality between threads and allocated pages. To improve memory locality, the initialization phase applies a first-touch policy to both planes using the same tiled distribution adopted during computation:

```
#pragma omp parallel for collapse(2) \
    schedule(static)
for (tiles...)
    data[...] = 0.0;
```

During the first initialization of the grid, each OpenMP thread writes to the tiles that it will later process. Under Linux's default first-touch memory policy, pages are allocated in the NUMA domain of the thread performing the first access. This ensures that computation threads primarily access local memory, reducing the overhead of remote NUMA accesses and improving memory bandwidth utilization.

To preserve memory locality during execution, the OpenMP runtime was configured with

```
export OMP_PLACES=cores
export OMP_PROC_BIND=close
```

The `OMP_PLACES=cores` setting binds threads to individual physical cores, while `OMP_PROC_BIND=close` places neighbouring threads on nearby cores

whenever possible. Combined with the first-touch initialization, these settings reduce thread migration and help maintain locality between threads and the memory pages allocated in their NUMA domain, improving the effective utilization of the memory hierarchy.

The computation of the total energy was also parallelised using OpenMP. Since the operation consists of a global summation over all grid points, the loop employs a `reduction(+)` clause, allowing each thread to accumulate a private partial sum that is automatically combined at the end of the parallel region. This avoids synchronization overhead while ensuring the correctness of the final result.

3.3 Thread Scaling

The OpenMP scaling experiments were performed on a single Orfeo EPYC node, using one MPI rank and increasing the number of OpenMP threads from 1 to 128. The node contains two AMD EPYC 7H12 processors, for a total of 128 cores distributed across eight NUMA domains.

The scalability was evaluated using the speedup and parallel efficiency:

$$S(p) = \frac{T_1}{T_p} \quad E(p) = \frac{S(p)}{p}$$

where T_p is the execution time using p threads.

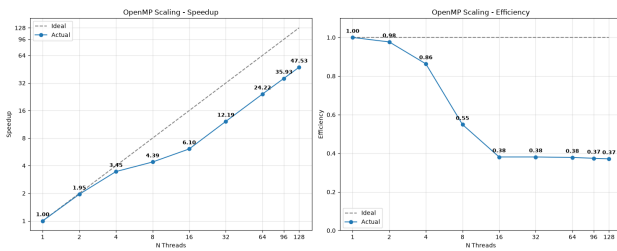


Figure 1: OpenMP Scaling: Speedup and Efficiency

3.4 Performance Discussion

The OpenMP scaling results show a strong improvement in execution time when increasing the number of threads, although the efficiency decreases as more cores are employed. Up to four threads, the computation scales almost linearly, achieving an efficiency above 85%. This indicates that, in this range, the available memory

bandwidth and computational resources are not yet saturated, allowing additional threads to effectively contribute to reducing execution time.

For higher thread counts, the speedup continues to increase but with diminishing returns. The efficiency decreases to approximately 38% beyond 16 threads and remains almost constant up to the full 128-core node. This behavior is consistent with the memory-bound nature of the problem. Since each update performs a limited number of floating-point operations compared to the amount of data accessed, the available memory bandwidth becomes the main performance bottleneck. Increasing the number of threads beyond the point where memory bandwidth is saturated cannot provide proportional improvements, as all threads compete for the same memory resources. Crossing into a new NUMA domain, however, adds an entirely independent memory controller, so the achievable bandwidth genuinely increases and speedup resumes.

The use of tiling, first-touch initialization, and thread affinity improves cache reuse and NUMA locality, allowing the application to exploit the available memory hierarchy more efficiently. However, these optimizations cannot remove the fundamental bandwidth limitation of the algorithm. The final configuration using 128 OpenMP threads achieves a speedup of approximately $47.5\times$, demonstrating substantial parallel acceleration while highlighting the scalability limits imposed by the underlying memory subsystem.

4 MPI Parallelisation

4.1 Domain Decomposition

To enable distributed-memory execution, the global computational domain is partitioned among the available MPI processes using a two-dimensional Cartesian decomposition. Each MPI rank is assigned a rectangular subdomain together with an additional halo layer storing the boundary values exchanged with neighbouring processes.

The process grid is chosen automatically according to the aspect ratio of the global domain in order to obtain subdomains that are close to square. Compared to a one-dimensional decomposition, this reduces the surface-to-volume ratio of each subdomain, thereby decreasing the amount of halo communication relative to the computational work-

load.

When the global domain cannot be divided evenly, the remaining rows or columns are distributed among the first processes along each dimension, ensuring that the workload differs by at most one grid point between neighbouring ranks.

4.2 Halo Exchange

At every iteration, each process requires the boundary values of its neighbouring subdomains before updating its own border cells. Consequently, one layer of halo cells is exchanged with the four neighbouring processes at every timestep. Processes located on the physical boundary communicate with `MPI_PROC_NULL`, allowing every rank to issue the same communication calls without requiring special-case logic.

North and south halo rows are contiguous in memory and can therefore be transmitted directly by passing a pointer to the corresponding row of the computational grid (zero-copy trick: `buffers[SEND][NORTH/SOUTH] = &data[IDX(1,1)]`). In contrast, east and west halo columns are stored with a stride and must first be packed into contiguous communication buffers before being exchanged. After reception, the column data are unpacked into the halo cells of the local subdomain.

To reduce the communication overhead, halo exchanges are performed using the non-blocking routines `MPI_Isend` and `MPI_Irecv`. All communication requests are posted before the stencil computation begins, allowing the interior of the domain to be updated while data transfers progress in the background. Since interior cells do not depend on halo values, this computation overlaps communication and partially hides the associated latency. Synchronization through `MPI.Waitall` is required only before updating the border cells, where the received halo values become necessary.

4.3 Communication Strategy

The stencil update is divided into two separate kernels: an interior update and a border update. This separation is fundamental for overlapping communication with computation, as the interior cells can be processed independently of the halo exchange, whereas the border cells require the newly received halo values. The application

initializes MPI using the `MPI_THREAD_FUNNELED` threading level, since all MPI communication is performed by the master thread while OpenMP is used exclusively for the computational kernels. Computation and communication times are measured separately using `MPI_Wtime`. At the end of each execution, both the maximum and the average time across all MPI processes are reported. The maximum represents the critical path that determines the overall execution time, while the average provides additional insight into the workload distribution among processes. Finally, the total energy of the simulation is obtained by summing the local energies computed by each process through an `MPI_Reduce` operation.

Global energy conservation is checked via `output_energy_stat`, which reduces each rank's local energy (from `get_total_energy`) with `MPI_Reduce/MPI_SUM` onto rank 0 for reporting.

5 Performance Evaluation

5.1 Process and Thread Configuration

The multi-node experiments were performed using a hybrid configuration of 8 MPI ranks per node, each spawning 16 OpenMP threads. A rankfile was used to pin each MPI rank to a distinct NUMA domain, while OpenMP threads were bound to the cores of that domain. This configuration was motivated by the single-node OpenMP scaling results: efficiency decreases rapidly up to 16 threads before stabilizing, indicating that a single NUMA domain reaches memory-bandwidth saturation. Assigning one MPI rank to each NUMA domain and giving it the 16 threads that domain owns lets every rank run at the best attainable OpenMP efficiency for that domain, while relying on MPI for additional domains. Moreover, it preserves memory locality while allowing the application to exploit the aggregate bandwidth of all eight NUMA domains on a node.

Strong scaling measures the reduction in execution time obtained by increasing the number of MPI processes while keeping the global problem size fixed. Weak scaling evaluates whether execution time remains constant when both the number of MPI processes and the problem size increase proportionally.

5.2 MPI Strong Scaling

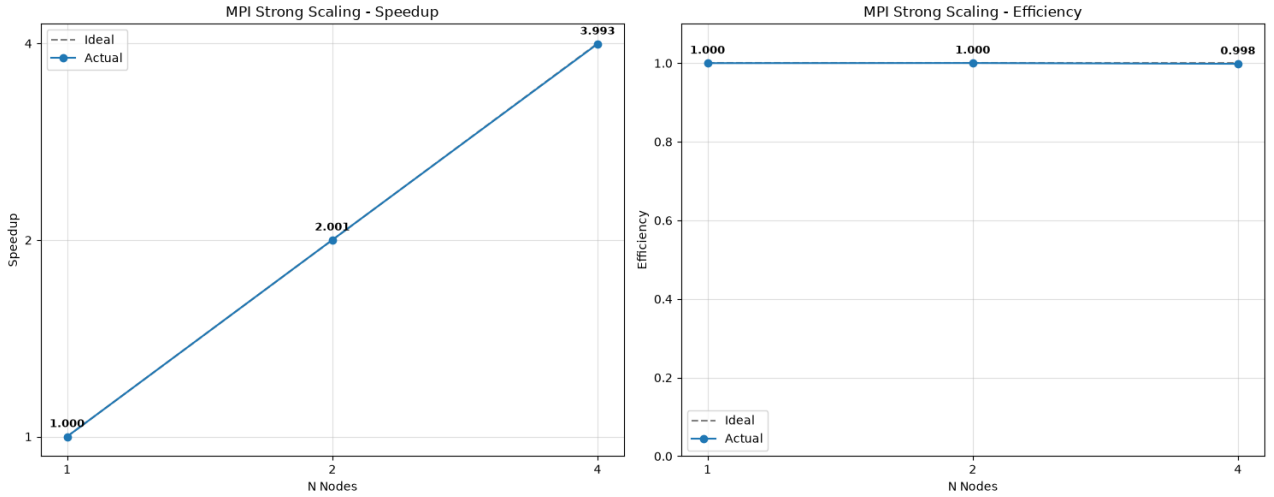


Figure 2: MPI Strong Scaling: Speedup and Efficiency

The plot reports speedup and efficiency (averaged over 3 repetitions) for the fixed 64000×64000 global grid, using 8, 16, and 32 MPI ranks (1, 2, and 4 nodes respectively): scaling is essentially ideal across the tested range. This is the direct payoff of the one-rank-per-NUMA-domain placement described above: each rank keeps operating on its own local subdomain with its own dedicated memory bandwidth, so adding ranks adds independent compute-and-bandwidth units rather than forcing existing ones to share. The split interior/border update combined with non-blocking `MPI_Isend/MPI_Irecv` overlaps the halo exchange with interior computation, so as the per-rank subdomain shrinks with increasing task count, communication cost remains sufficiently small to not significantly affect the observed scalability.

5.3 MPI Weak Scaling

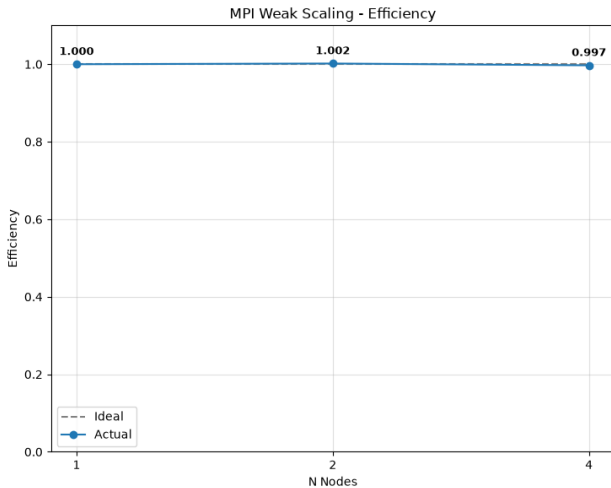


Figure 3: MPI Weak Scaling: Efficiency

The plot reports the weak-scaling results, where the grid side is increased proportionally to $\sqrt{N_{\text{nodes}}}$, starting from a 40000×40000 grid, so that the local subdomain size, and therefore the amount of work and memory traffic handled by each MPI rank, remains constant as the number of ranks grows. Computation time per rank should therefore stay constant if the implementation scales ideally, and this is exactly what is observed: execution time varies by less than 0.3% across the tested range (13.67s – 13.71s), giving efficiency indistinguishable from 1 within the noise of averaging 3 repetitions.

Nodes	Tasks	Grid side	Comp. time (s)	Efficiency
1	8	40000	13.668	1.0000
2	16	56576	13.643	1.0018
4	32	80000	13.707	0.9971

This result is the natural counterpart to the strong-scaling behaviour: in strong scaling the per-rank subdomain shrinks as more ranks are added, so the near-ideal speedup demonstrates that communication overlap successfully hides the halo exchange even as it becomes a proportionally larger fraction of a shrinking workload. In weak scaling, on the other hand, the per-rank subdomain is kept fixed by construction, so ideal efficiency confirms that no additional bottleneck emerges purely from involving more nodes. Together, the two experiments show that the one-rank-per-NUMA-domain placement scales cleanly along both axes: adding domains adds proportional work without eroding per-domain performance, and growing the total job while holding

per-domain work fixed leaves per-domain performance untouched.

5.4 Overall Performance Discussion

The optimization process followed the memory hierarchy of the target architecture. The roofline analysis showed that the five-point stencil is inherently memory-bound, motivating optimizations aimed at reducing memory traffic rather than arithmetic operations. Cache blocking significantly improved cache locality by increasing data reuse within the cache hierarchy, reducing the L1 data cache miss rate from 8.4% to approximately 4%.

The OpenMP scaling results then identified the memory subsystem as the primary limiting factor for shared-memory parallelism. Performance scaled efficiently up to the point where the memory bandwidth of a single NUMA domain became saturated, after which additional threads provided diminishing returns. This observation motivated the hybrid MPI/OpenMP configuration adopted for the distributed-memory experiments, assigning one MPI rank to each NUMA domain with 16 OpenMP threads per rank.

Finally, the MPI experiments demonstrated that this configuration scales efficiently across multiple nodes. Strong scaling remained close to ideal, indicating that the communication overhead introduced by halo exchanges was effectively mitigated through non-blocking communication and overlap with interior computation. Likewise, the weak-scaling results showed that execution time remained essentially constant as both the problem size and the number of processes increased proportionally, confirming that the implementation introduces negligible additional overhead when extending the computation to multiple nodes.

Overall, the combination of cache-aware serial optimizations, NUMA-aware OpenMP parallelisation, and hybrid MPI/OpenMP execution produced a scalable implementation that efficiently exploits both the shared-memory and distributed-memory resources of the Orfeo EPYC cluster.